Title: Process Creation and Control Using POSIX APIs

1. AIM

To study and implement process creation and control using POSIX system calls in Linux. To understand the working of `fork()`, `exec()` and `wait()` system calls to create, manage and coordinate the execution of parent and child processes.

2. THEORY

2.1 Introduction to Process

A process is an instance of a program that is currently being executed by the operating system. A process includes program instructions, data, memory, CPU registers and other resources required for execution.

The operating system manages processes and allocates resources to them.

Every process in Linux has a unique Process ID (PID), which helps the operating system identify and manage it.

Important process attributes include:

- Process ID (PID): Unique identifier of a process.
- Parent Process ID (PPID): Identifier of the process that created another process.
- Process State: Current execution state of the process.
- Program Counter: Stores the address of the next instruction.
- CPU Registers: Store temporary processing information.
- Memory Space: Contains program code, data and stack.

2.2 Process Creation

Process creation is the operation of generating a new process during program execution.

In Linux and Unix-based operating systems, the `fork()` system call is used to create a new process.

The process that calls `fork()` is called the parent process, and the newly created process is called the child process.

The child process initially receives a copy of the parent's process address space.

Syntax:

```
None
pid_t fork(void);
```

The return values of `fork()` are:

Return Value	Meaning
`-1`	Process creation failed
`0`	Executed in the child process
Positive value	Executed in the parent process; value is the child's PID

Example:

```
None
pid_t pid;

pid = fork();

if (pid == 0)
{
    printf("Child Process\n");
}
else if (pid > 0)
{
    printf("Parent Process\n");
}
else
```

```
{
    printf("Fork Failed\n");
}
```

2.3 Parent and Child Processes

A parent process is responsible for creating and managing child processes.

A child process is an independent execution flow created by its parent process.

Both processes execute independently after a successful `fork()` call.

Important characteristics:

1. Every process has a unique PID.
2. The child process receives a copy of the parent's address space.
3. Changes to ordinary variables in one process do not automatically change the corresponding variables in the other.
4. Parent and child processes may execute concurrently.
5. The operating system schedules both processes for CPU execution.

2.4 exec() System Call

The `exec()` family of system calls is used to replace the current process image with a new program.

After successful execution of an exec function, the previous program code is replaced by the new program.

The process ID remains the same because the existing process is replaced rather than creating another process.

Common functions in the exec family include:

Function	Description
`execl()`	Executes a program using a list of arguments

`execv()`	Executes a program using an argument array
`execlp()`	Searches for the executable using PATH
`execvp()`	Executes a program using an argument array and PATH
`execve()`	Executes a program with specified arguments and environment

Syntax:

```
None
execlp("program", "argument0", "argument1", (char *)NULL);
```

Example:

```
None
execlp("ls", "ls", "-l", (char *)NULL);
```

This command replaces the current process with the Linux `ls` program and displays directory contents in long format.

If an exec function executes successfully, it does not return to the original program.

If execution fails, it returns `-1`.

2.5 wait() System Call

The `wait()` system call is used by a parent process to wait until one of its child processes terminates.

It helps coordinate parent-child process execution.

Syntax:

```
None
pid_t wait(int *status);
```

The wait function provides the following benefits:

- Prevents the parent from proceeding before a child terminates.
- Collects the termination status of the child.
- Helps prevent zombie processes.
- Maintains proper execution order between parent and child.

Example:

```
None
int status;

wait(&status);
```

The parent process waits until a child process terminates.

2.6 waitpid() System Call

The `waitpid()` system call provides more control over waiting for child processes.

Syntax:

```
None
pid_t waitpid(pid_t pid, int *status, int options);
```

It allows the parent process to wait for a particular child process.

2.7 Process Identification Functions

Linux provides functions to obtain process identification information.

Function	Description

`getpid()`	Returns the current process ID
`getppid()`	Returns the parent process ID
`fork()`	Creates a new process
`wait()`	Waits for child process termination
`waitpid()`	Waits for a specific child process

2.8 Process Termination

A process terminates when it completes its execution or when it is explicitly terminated.

The `exit()` function is used to terminate a process and return an exit status.

Example:

```
None
exit(0);
```

Here, `0` indicates successful termination.

2.9 Zombie Process

A zombie process is a process that has completed execution but still has an entry in the process table because its parent has not yet collected its termination status.

The parent process can collect the child's status using `wait()` or `waitpid()`.

2.10 Orphan Process

An orphan process is a child process whose parent terminates before the child completes execution.

The operating system typically reassigns the orphan process to a system process such as `init` or a subreaper.

2.11 Process Execution Flow

The following sequence explains the working of process creation and control:

1. The parent process starts execution.
2. The parent calls `fork()`.
3. A child process is created.
4. The child executes a new program using `exec()`.
5. The parent calls `wait()` and waits for the child.
6. The child completes its execution.
7. The parent resumes execution.

3. CODE

Program: Process Creation and Control Using fork(), exec() and wait()

File name: `process_control.c`

```
None
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent Process Started\n");

    // Display parent process ID
    printf("Parent PID: %d\n", getpid());
```

```
// Create child process
pid = fork();

if (pid < 0)
{
    // Fork failed
    perror("Fork Failed");
    exit(1);
}

else if (pid == 0)
{
    // Child process
    printf("\nChild Process Started\n");

    printf("Child PID: %d\n", getpid());

    printf("Child Parent PID: %d\n", getppid());

    printf("Executing ls command using exec()\n");

    // Replace child process with ls command
    execlp("ls", "ls", "-l", (char *)NULL);

    // Executes only if exec fails
    perror("Exec Failed");

    exit(1);
}

else
{
    // Parent process
    printf("\nParent is waiting for child process\n");
```

```
        // Wait for child process to finish
        wait(&status);

        if (WIFEXITED(status))
        {
            printf("\nChild terminated successfully\n");

            printf("Child Exit Status: %d\n",
                   WEXITSTATUS(status));
        }
        else
        {
            printf("\nChild terminated abnormally\n");
        }

        printf("Parent Process Completed\n");
    }

    return 0;
}
```

Explanation of the Program

1. Header Files

- `stdio.h`: Provides input and output functions.
- `stdlib.h`: Provides the `exit()` function.
- `unistd.h`: Provides POSIX functions such as `fork()`, `exec()` and `getpid()`.
- `sys/types.h`: Defines process-related data types.
- `sys/wait.h`: Provides `wait()` and process-status macros.

2. Process Creation

```
None
pid = fork();
```

This creates a new child process.

3. Child Process

```
None
if (pid == 0)
```

This block executes in the child process.

4. Process Replacement

```
None
execlp("ls", "ls", "-l", (char *)NULL);
```

The child process executes the Linux `ls -l` command.

5. Parent Process

```
None
else
{
    wait(&status);
}
```

The parent waits until the child process finishes.

6. Exit Status

```
None
WIFEXITED(status)
WEXITSTATUS(status)
```

These macros check whether the child terminated normally and retrieve its exit status.

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.

- Terminal: Linux Terminal.
- Required libraries: Standard POSIX system headers.

Step 1: Open Linux Terminal

Press:

```
None
Ctrl + Alt + T
```

Step 2: Check GCC Compiler

Execute:

```
None
gcc --version
```

If GCC is not installed, install it using:

```
None
sudo apt update
sudo apt install build-essential
```

Step 3: Create a Practical Directory

```
None
mkdir Process_Control
cd Process_Control
```

Step 4: Create the C Program

```
None
nano process_control.c
```

Copy and paste the complete C program given above.

Save the file:

- Press `Ctrl + O`.

- Press Enter.
- Press `Ctrl + X`.

Step 5: Compile the Program

Execute:

```
None
gcc process_control.c -o process_control
```

If there are no compilation errors, the executable file will be created.

Step 6: Run the Program

Execute:

```
None
./process_control
```

Step 7: Observe the Output

The program displays:

- Parent process ID.
- Child process ID.
- Parent process ID of the child.
- Directory listing generated by the `ls -l` command.
- Child process termination status.
- Parent process completion message.

Step 8: Verify Process Information

You can also use the following commands:

```
None
ps
```

To display detailed process information:

```
None
ps -ef
```

To display the current process ID:

```
None
echo $$
```

5. RESULT

The process creation and control operations using POSIX system calls were studied and implemented.

The following operations were demonstrated:

1. Created a child process using the `fork()` system call.
2. Identified parent and child processes using `getpid()` and `getppid()`.
3. Replaced the child process with another program using `execlp()`.
4. Used the `wait()` system call to synchronize parent and child execution.
5. Retrieved the child process termination status.
6. Observed parent-child process execution and coordination.

6. CONCLUSION

In this practical, process creation and control using POSIX system calls were studied and implemented in the Linux environment. The `fork()` system call was used to create child processes, while the `exec()` family was used to execute new programs. The `wait()` system call was used to synchronize parent and child process execution. Thus, the practical provided an understanding of process management, process execution and synchronization in Unix-like operating systems.